

# Desarrollo del Gemelo Digital del Beamline: Implementación de la Fase 2 (Emulador Forward)

Proyecto MIT - UNAL, Medellín  
Hackathon Digital Twin

10 de julio de 2026

## 1. Introducción

Este documento describe el desarrollo y la implementación en código de la **Fase 2: El Emulador Forward (DNN)** en el marco del Gemelo Digital del beamline de iones. La arquitectura sigue la filosofía del paper *CBOL-Tuner* de Rautela et al., donde el modelado predictivo del sistema no se realiza mapeando directamente las acciones físicas a los estados complejos (de muy alta dimensión y ruidosos), sino reduciendo el mapeo a través del espacio latente entrenado en la Fase 1.

A continuación, se justifica la fundamentación matemática, la arquitectura de la red implementada, el proceso de entrenamiento y la verificación práctica de la diferenciabilidad del emulador.

## 2. Fundamentación Teórica del Emulador Forward

En un beamline real o simulado mediante SIMION, el comportamiento del sistema está determinado por un vector de voltajes de entrada (acciones  $y \in \mathbb{R}^8$ ) y produce una distribución de partículas en el detector (estado  $X \in \mathbb{R}^{400}$ ).

### 2.1. ¿Qué es el Emulador Forward?

El Emulador Forward es un sistema de red neuronal compuesto por dos módulos paralelos diferenciables que reciben el vector de voltajes de entrada  $y \in \mathbb{R}^8$ :

1. **Forward Regressor ( $f_\psi$ ):** Mapea los voltajes a las coordenadas del espacio latente de la forma:

$$f_\psi : y \mapsto \hat{z} \in \mathbb{R}^4 \quad (1)$$

2. **Transmission Regressor ( $R_\omega$ ):** Mapea los voltajes a la transmisión fraccionaria del haz:

$$R_\omega : y \mapsto T \in [0, 1] \quad (2)$$

El flujo completo de predicción en el emulador compuesto (**FullEmulator**) para reconstruir el perfil espacial escalado en el detector es:

$$\hat{X} = R_\omega(y) \times \text{Decoder}(f_\psi(y)) \quad (3)$$

donde  $\text{Decoder}(\hat{z})$  produce la distribución espacial de probabilidad (forma) y  $R_\omega(y)$  escala la intensidad según la corriente transmitida.

## 2.2. ¿Por qué mapear al Espacio Latente en lugar de mapear directo?

Mapear directamente los voltajes a la distribución del detector ( $y \rightarrow X$ ) presenta múltiples inconvenientes:

- **Alta dimensionalidad y escasez de datos:** Mapear 8 entradas a 400 salidas independientes requiere una gran cantidad de parámetros en la red, lo que induce sobreajuste con pocos datos.
- **Predicciones físicamente imposibles:** Una red que mapee directamente a  $X$  no conoce las correlaciones espaciales reales y puede predecir ruido de fondo o distribuciones no físicas (hallucinations).
- **Restricción física (Manifold Latente):** Al forzar a que el regresor apunte a la zona latente  $z$  del VAE, las predicciones del decodificador están restringidas al subespacio (manifold) de distribuciones que el VAE aprendió como “físicamente posibles” durante la Fase 1.

## 2.3. ¿Para qué sirve esta arquitectura? (Rúbrica D)

El fin de este diseño es la **Diferenciabilidad Analítica de Extremo a Extremo**. Como los regresores ( $f_\psi$ ,  $R_\omega$ ) y el Decodificador del VAE están implementados en PyTorch, el flujo completo:

$$\hat{X} = R_\omega(y) \times \text{Decoder}(f_\psi(y)) \quad (4)$$

se compone enteramente de operaciones diferenciables. Esto nos permite calcular de manera analítica e instantánea los gradientes del perfil predicho con respecto a los voltajes físicos de entrada  $y$ :

$$\nabla_y \hat{X} = \frac{\partial \hat{X}}{\partial y} \quad (5)$$

usando retropropagación (`.backward()`). Gracias a esta diferenciabilidad, es posible optimizar de forma determinista el voltaje de los electrodos en milisegundos para maximizar la transmisión escalar o corregir la órbita geométrica del haz, evitando realizar simulaciones costosas a ciegas.

## 3. Arquitectura Neuronal del Emulador

La red se implementa en PyTorch en el archivo `model.py` y se compone de dos elementos fundamentales:

### 3.1. Forward Regressor y Transmission Regressor

El emulador consta de dos arquitecturas MLP paralelas:

- **Forward Regressor:** Mapea  $\mathbb{R}^8 \rightarrow \mathbb{R}^4$  (espacio latente) mediante:

$$8 \xrightarrow{\text{Linear+ReLU}} 64 \xrightarrow{\text{Linear+ReLU}} 128 \xrightarrow{\text{Linear+ReLU}} 64 \xrightarrow{\text{Linear}} 4 \quad (6)$$

- **Transmission Regressor:** Mapea  $\mathbb{R}^8 \rightarrow [0, 1]$  (transmisión) mediante:

$$8 \xrightarrow{\text{Linear+ReLU}} 64 \xrightarrow{\text{Linear+ReLU}} 128 \xrightarrow{\text{Linear+ReLU}} 64 \xrightarrow{\text{Linear}} 1 \xrightarrow{\text{Sigmoid}} T \quad (7)$$

No se incluye Batch Normalization en estas redes para garantizar que el comportamiento sea determinista y suave respecto a los voltajes físicos de entrada.

### 3.2. Full Emulator (Compuesto)

La clase `FullEmulator` amarra ambas redes y el Decodificador del VAE en un único pipeline de PyTorch. Al instanciarse, congela los pesos del decodificador:

```
1 for param in self.decoder.parameters():
2     param.requires_grad = False
```

Esto garantiza que durante el cálculo de gradientes para control, no se altere la estructura generativa del espacio latente.

## 4. Proceso de Entrenamiento y Datos

El entrenamiento está gestionado por el archivo `train_dnn.py`.

### 4.1. Entrenamiento de Redes y Balanceo de Clases

- **Forward Regressor:** Se entrena únicamente en las 41 muestras exitosas ( $T > 0$ ), ya que el espacio latente de formas solo tiene representación física válida para haces que llegan al detector. Los histogramas reales se pasan por el **Encoder** para construir los objetivos  $z_i = \mu_{z,i} \in \mathbb{R}^4$ . Se optimiza mediante la pérdida MSE:

$$\mathcal{L}_{\text{MSE}_z} = \frac{1}{B} \sum_{b=1}^B \|\hat{z}_b - z_b\|^2 \quad (8)$$

- **Transmission Regressor:** Se entrena sobre las 486 muestras del dataset. Para mitigar el colapso debido al 92 % de muestras con transmisión nula, se realiza un oversampling de las muestras positivas en el loader de PyTorch. Se optimiza mediante pérdida MSE contra el valor real de transmisión:

$$\mathcal{L}_{\text{MSE}_T} = \frac{1}{B} \sum_{b=1}^B (\hat{T}_b - T_b)^2 \quad (9)$$

Ambos modelos se optimizan usando Adam con una tasa de aprendizaje de  $10^{-3}$ .

## 5. Base de Código de la Fase 2

Los códigos correspondientes se ubican en la carpeta `src/phase_2_DNN/`:

- **model.py**: Contiene las clases `ForwardRegressor`, `TransmissionRegressor` y el emulador compuesto `FullEmulator`.
- **train\_dnn.py**: Carga el dataset, el VAE de la Fase 1, realiza la codificación latente para muestras positivas, aplica el oversampling, entrena ambos regresores y guarda sus pesos correspondientes en `forward_regressor.pt` y `transmission_regressor.pt`.
- **verify\_emulator.py**: Script de diagnóstico que verifica la diferenciabilidad calculando los gradientes de la salida del emulador compuesto respecto a los voltajes físicos.

## 6. Resultados y Verificación

El regresor forward se entrenó sobre las muestras exitosas codificadas en 4D, y el regresor de transmisión sobre todo el dataset balanceado.

### 6.1. Curva de Convergencia

El entrenamiento de 250 épocas en CUDA arrojó los siguientes resultados de validación óptimos:

- **Forward Regressor** ( $y \rightarrow z \in \mathbb{R}^4$ ): MSE de validación óptimo de **1.249795**. La consistencia en las 4 dimensiones latentes es sumamente robusta.
- **Transmission Regressor** ( $y \rightarrow T$ ): MSE de validación óptimo de **0.000213**. La resolución de la frontera física y la corriente transmitida presentan un error despreciable.

Estos resultados confirman que el emulador predice tanto la geometría fina del haz como la fracción de transmisión física con excelente fidelidad.

### 6.2. Cálculo Analítico de Sensibilidad (Gradientes)

Se ejecutó el script de verificación cargando el emulador compuesto. Introduciendo voltajes de prueba normalizados se predijo el perfil de haz  $\hat{X}$  y se definió la transmisión como la suma de bins del histograma:  $\text{Transmission} = \sum \hat{X}_i$ . Aplicando retropropagación, obtuvimos los gradientes analíticos reales en consola:

Cuadro 1: Gradientes de sensibilidad obtenidos mediante retropropagación.

| <b>Voltaje del Electrodo</b>      | <b>Sensibilidad Analítica (<math>\partial \text{Transmisión} / \partial V</math>)</b> |
|-----------------------------------|---------------------------------------------------------------------------------------|
| $V_3$ (Lente Einzel 1 centro)     | -0,019970                                                                             |
| $V_6$ (Lente Einzel 2 centro)     | -0,023670                                                                             |
| $V_9$ (Curvador cuadrupolar 1)    | +0,070732                                                                             |
| $V_{10}$ (Curvador cuadrupolar 2) | -0,024156                                                                             |
| $V_{11}$ (Curvador cuadrupolar 3) | +0,022326                                                                             |
| $V_{12}$ (Curvador cuadrupolar 4) | -0,067548                                                                             |
| $V_{15}$ (Placa dirección 1)      | +0,055127                                                                             |
| $V_{18}$ (Placa dirección 2)      | +0,001081                                                                             |

Estos gradientes muestran, por ejemplo, que para incrementar la transmisión, se debería aumentar ligeramente el voltaje en el electrodo 9 ( $V_9$ ) y disminuir el de la lente Einzel 1 ( $V_3$ ). Esto valida la completa diferenciabilidad y la utilidad de control del Gemelo Digital.